

SmartStock

Inventory & Billing Management System

PROJECT REPORT

Student Name	NIRAN KRISHNA A R
Registration No.	25BAI10789
Course	Programming in Java
Submission Type	Build Your Own Project (Evaluated Project)
Platform	VITyarthi
Date	September 2026

A practical Java project designed around the everyday needs of a small retail store.

1. Introduction

SmartStock is a command-line Java application created to make three common shop activities easier to manage: keeping track of products, preparing customer bills, and checking sales performance. I designed the project as a small but realistic business application so that the Java concepts learned in the course could be used in a practical setting.

The application does not depend on a database server or a large framework. It can be compiled and run using a normal JDK, while the application's data is stored in readable CSV files inside the **data/** folder. This keeps the project simple to set up and also makes it easy to inspect what the program is doing.

2. Problem Statement

A small retail shop may use notebooks, spreadsheets, or separate tools for stock, billing, and sales records. When these systems are disconnected, it becomes easy to make mistakes: stock counts may not match the actual stock, a bill may contain unavailable items, and the owner may have little idea which products are selling well.

SmartStock brings these tasks together in one console application. The main idea is straightforward: the operator should be able to see the current inventory, create a bill, update stock automatically after a sale, and review basic sales information without needing a separate database or network connection.

3. Functional Requirements

The project is divided into three modules so that each part has a clear purpose.

- **Inventory Management:** Add, update, delete, and display products. Products with five or fewer units are marked as low stock.
- **Billing & Invoice Generation:** Enter a customer and cart items, check availability, create a unique invoice, deduct sold quantities, and store the completed sale.
- **Sales Reporting & Analytics:** Calculate total revenue, identify top-selling products, and show low-stock information using current inventory and sales records.

4. Non-Functional Requirements

Requirement	How SmartStock handles it
Performance	Products are kept in a HashMap, giving fast average O(1) lookups. CSV files are mainly written when data changes.
Reliability	The complete cart is validated before any stock is deducted, preventing half-completed bills.
Usability	A numbered console menu guides the operator and error messages explain what needs to be corrected.
Maintainability	The code is separated into model, service, exception, and utility packages.
Error handling	Three checked exceptions handle invalid input, missing products, and insufficient stock.
Logging	Important actions are timestamped in data/app.log as well as shown in the console.
Input safety	Identifiers and numeric values are checked before being used; blank and negative values are rejected.
Resource use	The application has no external dependencies, threads, or network calls.

5. System Architecture

The application follows a simple layered structure. The console menu in **Main.java** receives input from the store operator and passes the work to the appropriate service. The services use model classes for the data and a common file-storage utility for persistence. Logging runs across the application so that important actions can be traced later.

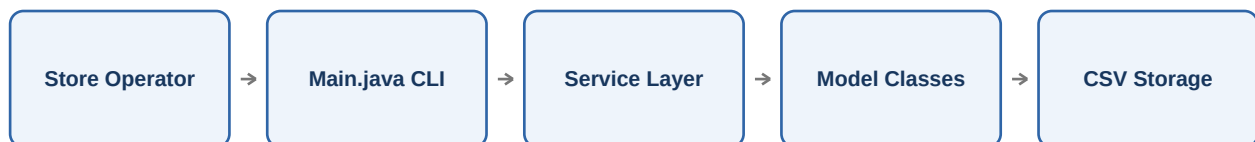


Figure 1: Simplified system architecture

This separation keeps the program easier to understand: the inventory logic stays in `InventoryService`, billing logic stays in `BillingService`, and reporting logic stays in `ReportService`.

6. Design Diagrams

6.1 Use Case Diagram

The main actor is the **Store Operator**. From the console menu, the operator can view inventory, add or update products, delete products, create an invoice, and view sales and inventory reports. Low-stock alerts appear as part of the reporting process.

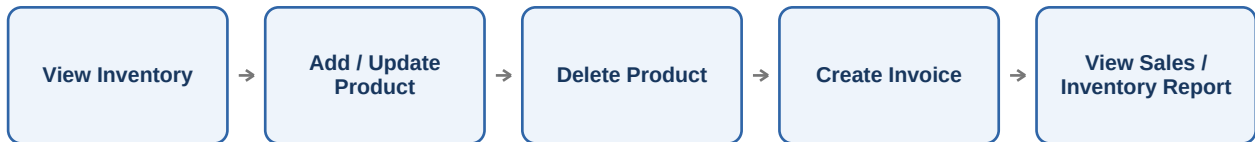


Figure 2: Main use cases represented as a workflow

6.2 Billing Workflow

Billing is the most important workflow from a reliability point of view. Before changing the inventory, the program checks every item in the cart. Only after every item passes validation does it deduct the stock and save the sale.

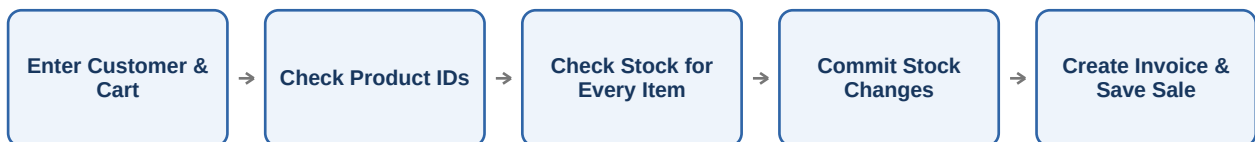


Figure 3: Billing workflow — validate first, then commit

6.3 Class-Level Design

The project uses model classes such as Product, Invoice, and InvoiceItem; service classes for the three functional modules; checked exception classes for expected failures; and utility classes for CSV storage and logging.



Figure 4: Major class/package relationships

6.4 Sequence Diagram — Create Invoice

When an invoice is created, Main collects the input and asks BillingService to process it. BillingService checks the required products and stock through InventoryService. If validation succeeds, stock is updated and the invoice is written to the sales ledger.

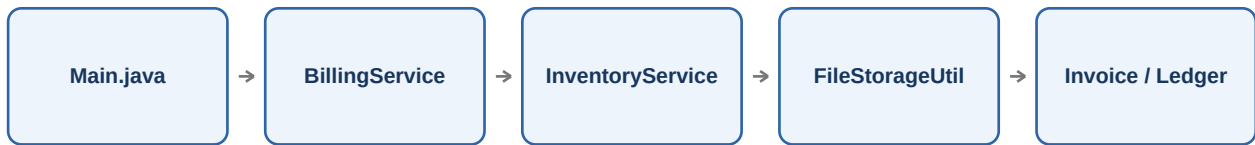


Figure 5: Simplified sequence of invoice creation

7. Design Decisions & Rationale

- **CSV instead of a database:** This keeps the application dependency-free and easy to run from a terminal.
- **Validate-then-commit billing:** Every cart line is checked before stock is changed, so an out-of-stock item cannot leave the inventory partially updated.
- **Checked custom exceptions:** InvalidInputException, ProductNotFoundException, and InsufficientStockException make expected failure cases explicit.
- **Service-per-module design:** Each service focuses on one responsibility instead of duplicating business logic.
- **Simple test runner:** The project uses plain assertions and a main() method, avoiding the need for JUnit or another external testing tool.

8. Implementation Details

The project is written in plain Java and was developed and verified using JDK 21. The source is organized into five packages, with each package having a focused responsibility.

Package	Classes	Responsibility
com.smartstock	Main	Console menu, input parsing, and top-level error handling
com.smartstock.model	Product, Invoice, InvoiceItem	Data objects and CSV serialization/deserialization
com.smartstock.service	InventoryService, BillingService, ReportService	Core application logic for the three modules
com.smartstock.exception	InvalidInputException, ProductNotFoundException, InsufficientStockException	Domain-specific checked exceptions
com.smartstock.util	FileStorageUtil, AppLogger	CSV persistence and logging
com.smartstock.test	SmartStockTest	Assertion-based test suite

The application stores products in **data/products.csv** and completed sales in the append-only **data/sales_ledger.csv**. Application events are recorded in **data/app.log**.

Command-line compilation and execution:

```
find src -name "*.java" > sources.txt
javac -d out @sources.txt
java -cp out com.smartstock.Main
```

9. Screenshots / Results

The original project report records an actual terminal run of the compiled application. The examples below show the inventory screen, a generated invoice, and the resulting analytics. These outputs demonstrate the three main modules working together.

```
===== SmartStock Menu =====
1. View all products
2. Add product
3. Update product
4. Delete product
5. Create invoice (sell products)
6. View sales & inventory report
0. Exit
Choose an option:
-- Current Inventory --
P001  Wireless Mouse      Electronics  Rs.499.00   Qty:40
P002  Mechanical Keyboard Electronics  Rs.2499.00  Qty:15
P003  Notebook (A5)       Stationery   Rs.60.00    Qty:200
P004  USB-C Cable 1m      Electronics  Rs.249.00   Qty:4      [LOW STOCK]
P005  Desk Lamp           Home        Rs.899.00   Qty:8
===== SmartStock Menu =====
1. View all products
2. Add product
3. Update product
4. Delete product
5. Create invoice (sell products)
6. View sales & inventory report
0. Exit
```

Figure 6: Seeded inventory shown in the terminal; P004 is flagged as low stock.

```
SmartStock -- Invoice Receipt (Menu Option 5)

=====
INVOICE: INV1000
Customer : Meera
Date    : 2026-09-16 10:36:19
-----
Wireless Mouse      x2   @ Rs.499.00   = Rs.998.00
Sticky Notes        x10  @ Rs.45.00    = Rs.450.00
-----
GRAND TOTAL: Rs.1448.00
=====
```

Figure 7: Generated invoice receipt after selling to customer 'Meera' (Menu Option 5)

Figure 7: Example invoice generated for customer "Meera".

9. Screenshots / Results — Continued

The analytics screen combines the current inventory with the recorded sales. The original run shows that a completed sale is reflected in the reporting output, which is the main purpose of keeping a separate append-only sales ledger.

```
Total Revenue: Rs.1448.00
Products in catalogue: 6
-- Top Selling Products --
P006      10 unit(s) sold
P001       2 unit(s) sold
-- Low Stock Alerts (<= 5 units) --
P004  USB-C Cable 1m      Electronics  Rs.249.00  Qty:4  [LOW ST0
=====
```

Figure 8: Sales & inventory analytics after the sale (Menu Option 6)

Figure 8: Sales and inventory analytics from the recorded project run.

10. Testing Approach

Testing was kept deliberately simple so that it can be run with only the standard Java compiler and runtime. SmartStockTest uses assertions and separate test data under data/test-run/, preventing tests from changing the application's normal data.

- addAndRetrieveProduct — verifies that an added product can be retrieved with the same values.
- duplicateProductRejected — confirms that reusing a product ID raises InvalidInputException.
- negativePriceRejected — checks that negative prices are rejected.

- `productNotFoundOnMissingId` — checks the expected `ProductNotFoundException` for an unknown ID.
- `invoiceReducesStock` — verifies stock deduction and the invoice grand total after a valid sale.
- `insufficientStockRejected` — confirms that insufficient stock raises an exception and leaves stock unchanged.
- `reportRevenueAggregation` — checks revenue calculation across multiple invoices.
- `lowStockDetection` — checks that low-stock products are reported while well-stocked products are not.

Test result: the project report records all 8 assertions passing when `SmartStockTest` is run from the command line.

11. Challenges Faced

- **Handling multi-item billing safely:** An early approach could deduct stock as each cart line was processed. If a later line failed, the inventory could be left partially changed. The final design solves this with two passes: validate everything first, then make the changes.
- **Storing structured invoice data in CSV:** An invoice can contain a variable number of items, which does not fit naturally into one flat CSV row. The project therefore packs the line items into a delimited field while keeping the ledger append-only.
- **Keeping the project dependency-free:** Without JUnit or a build tool, the project needed a small custom assertion-based test runner. This required extra work but keeps the whole application runnable from a normal command line.

12. Learnings & Key Takeaways

- Custom checked exceptions make expected failures clearer and easier to handle.
- Separating model, service, exception, and utility code makes the project easier to navigate and test.
- A carefully designed flat-file format can provide useful persistence for a small application without the operational overhead of a database.
- Writing tests without a framework helped identify the exact behaviors that matter, especially keeping inventory unchanged after a rejected sale.

13. Future Enhancements

- Replace CSV persistence with an embedded SQLite database through JDBC while keeping the service layer largely unchanged.
- Add a JavaFX or web interface without rewriting the core business services.
- Support multiple stores or branches and introduce role-based accounts such as cashier and manager.
- Allow low-stock thresholds to be configured by product category and add automated email/SMS notifications.
- Extend billing with discounts, tax calculation, and multi-currency support.

14. References

- Oracle — Java Tutorials: Collections, Exceptions, and I/O.
- Oracle — Java Platform, Standard Edition Documentation.
- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.

Project: SmartStock — Inventory & Billing Management System

Student: NIRAN KRISHNA A R

Registration No.: 25BAI10789

Project at a Glance

SmartStock is a small Java application built to solve a simple real-world problem: keeping a shop's stock, billing, and sales information in one place. The project focuses on practical Java programming rather than adding unnecessary technology.

What I implemented

- Inventory CRUD operations with low-stock identification.
- Customer billing with invoice generation and automatic stock deduction.
- Atomic validation so failed sales do not partially change inventory.
- CSV-based persistence and application logging.
- Sales revenue, top-selling products, and low-stock reporting.
- Custom checked exceptions and an 8-test assertion-based test suite.

Student Details

Name	NIRAN KRISHNA A R
Registration No.	25BAI10789
Course	Programming in Java
Project	SmartStock